A CLOUD COMPUTING

PROJECT REPORT

on

SENTINEL: An Intelligent Monitoring Dashboard

Submitted by:

Vaibhaav Tiwari (230599)

Rohit Singh (230782)

Vikas Kumar (230816)

Shuvam Bhatt (230457)

under mentorship of

Dr. Harsh Taneja

(Assistant Professor)

BML Munjal University

Department of Computer Science Engineering

School of Engineering and Technology

BML MUNJAL UNIVERSITY, GURUGRAM (INDIA)

November 2025

Problem Statement

As digital systems grow more complex, teams increasingly depend on dashboards to monitor system status, performance metrics, and activity logs. However, most existing dashboard solutions come with serious limitations, such as:

- Many are slow to load because they're built on heavy frameworks packed with features you don't even need.
- Their interfaces often feel bulky and rigid, making it hard to adjust the design or layout to match your project.
- Setting them up can be surprisingly complicated, which is a problem when you just want something simple that works.
- They don't always behave well across different devices, leading to poor responsiveness.

Because of these limitations, traditional dashboards aren't ideal for students, small teams, or developers who need something lightweight, fast, and easy to modify.

Sentinel was created to solve this exact problem - offering a quick, responsive, and highly customizable monitoring dashboard built with Vite and Tailwind CSS. It loads instantly, adapts easily, and gives developers a clean foundation they can build on without unnecessary complexity.

Objectives

The main objectives of Sentinel are:

1. To build a **lightweight and fast** dashboard using Vite and modern JavaScript tooling.
2. To design a **clean and customizable UI** using Tailwind CSS.
3. To create a project structure that is **easy to scale, modify, and deploy**.
4. To implement seamless integration with backend APIs using modular architecture.
5. To ensure the dashboard remains **high-performance** during development and production.

Workflow / Design

The overall workflow of Sentinel is structured into the following phases:

1. Project Initialization

- Setting up the Node.js environment
- Creating the Vite project
- Installing and configuring Tailwind CSS, PostCSS, and Autoprefixer
- Initializing project directory and dependencies

2. UI/UX Design

- Building the base layout using `index.html`
- Applying Tailwind CSS utility classes for styling

- Ensuring a responsive design that adapts to all screen sizes

3. Frontend Architecture

- Structuring code into modular JavaScript components
- Creating reusable UI sections like cards, status panels, and dashboards
- Organizing assets, configurations, and styles for scalability

4. Google Cloud Integration

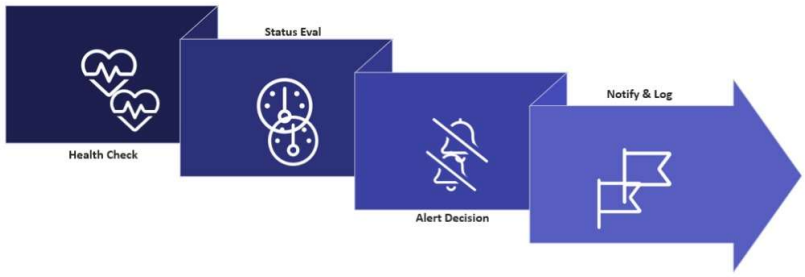
- Connecting the dashboard with Google Cloud for data handling
- Using Google Cloud services for secure storage, data retrieval, or API communication
- Configuring OAuth credentials and environment variables (kept private)
- Ensuring smooth interaction between frontend and Google Cloud backend services

5. Styling & Customization

- Customizing the Tailwind theme, colors, spacing, and typography
- Using PostCSS processing and Autoprefixer for cross-browser compatibility
- Maintaining a clean and consistent visual system throughout the dashboard

6. Build & Optimization

- Using Vite's optimized production build system
- Bundling and minifying project assets
- Running build previews to validate performance
- Preparing the dashboard for deployment or cloud-hosted environments



Methodology

1. Frontend Development

- Used **Vite** for fast development, hot-reload, and optimized builds
- Structured the project into **modular components** for scalability
- Ensured clean separation of UI, logic, and configurations

2. Styling System

- Implemented **Tailwind CSS** for utility-based styling
- Reduced custom CSS by using predefined utility classes
- Maintained consistent spacing, layout, and responsive design

3. Google Cloud Configuration

- Integrated Google Cloud for **secure authentication** and data management
- Used Google Cloud OAuth services for connecting dashboard features
- Managed sensitive environment variables securely (kept outside GitHub)
- Set up API endpoints / cloud services for backend processing

4. Connectivity Layer

- Dashboard communicates with Google Cloud through **REST APIs**
- Implemented client-side requests to fetch or send cloud data
- Ensured smooth integration so the dashboard works as a **cloud-connected system**, not just a static UI

5. Testing and Deployment

- Tested the system locally using **Vite's preview server**
- Verified UI responsiveness across devices
- Optimized build output using Vite's production bundling
- Prepared the project for deployment on cloud or hosting platforms

Results

- The Sentinel dashboard successfully delivers a **fast and lightweight monitoring interface** using Vite's optimized development and build system.
- The UI loads **almost instantly**, with smooth navigation and responsive layout across desktop and mobile screens.
- Tailwind CSS allowed the creation of a **clean, customizable, and modern interface**, reducing the need for bulky CSS files.
- Google Cloud integration was implemented to support secure backend connectivity, enabling the dashboard to interact with cloud services as needed.
- The final build is **stable, scalable, and easy to modify**, making it suitable for further development or real-world deployment.

- Overall, the system demonstrates strong performance, modular code structure, and readiness for integration with real-time data sources.

6. Screenshots

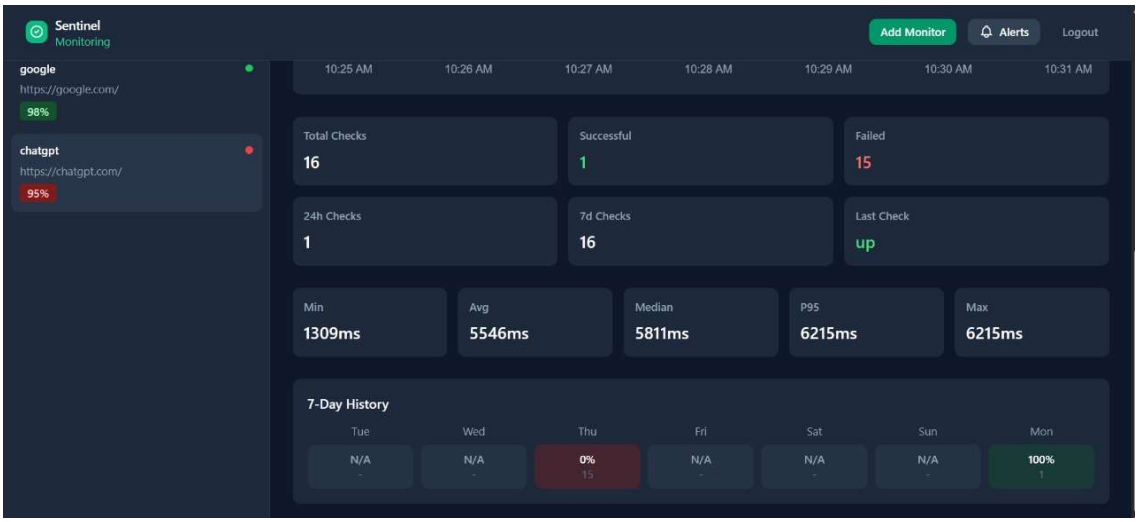


Fig.1

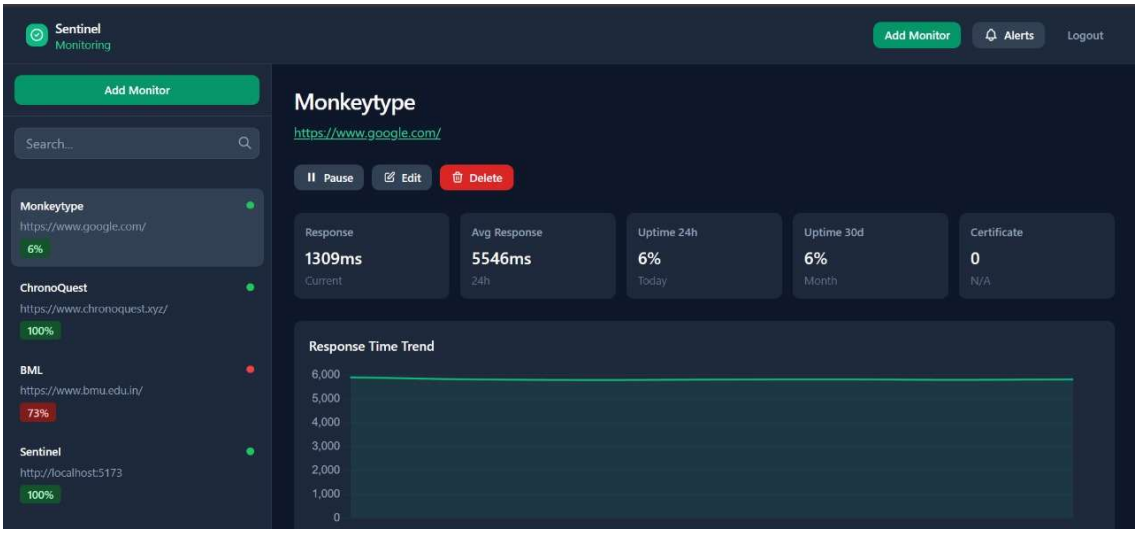


Fig.2

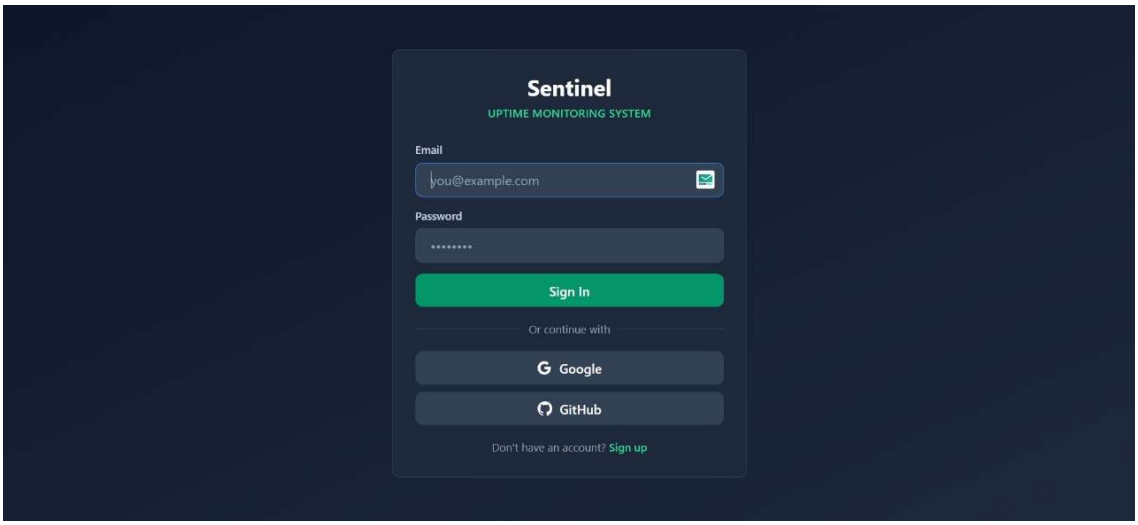


Fig.3

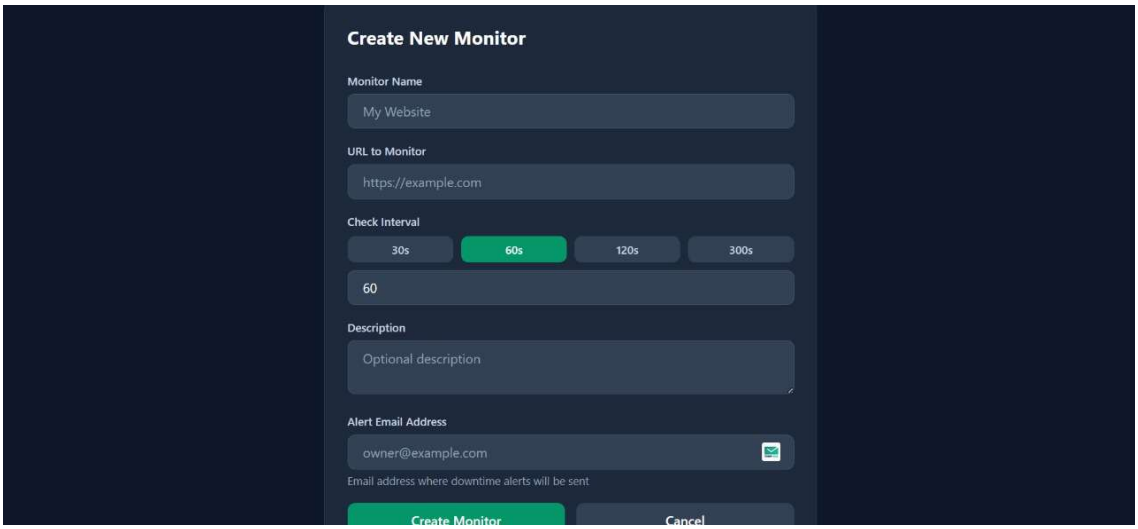


Fig.4

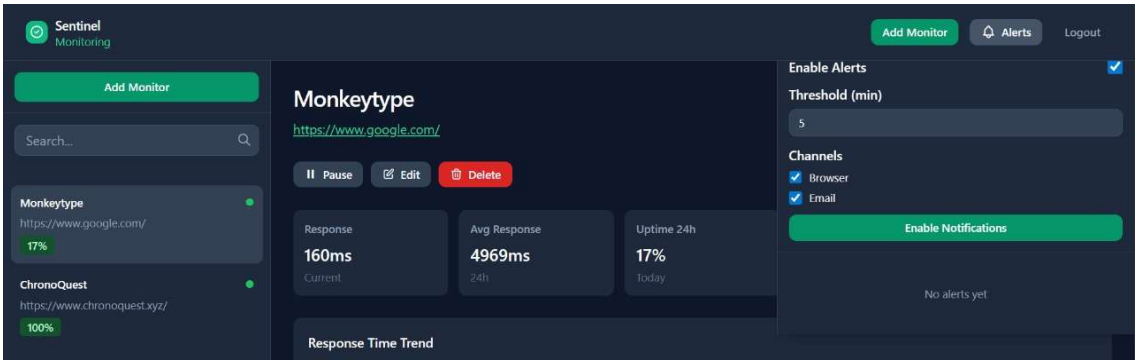


Fig.5

Conclusion & Future Scope

Conclusion

Sentinel achieves the goal of providing a **high-speed, customizable, and cloud-ready monitoring dashboard** without the complexity of traditional frameworks.
By combining Vite, Tailwind CSS, and Google Cloud, the project offers:

- instant load times
- flexible and modern UI design
- secure and efficient cloud connectivity
- a simple and expandable codebase

This makes Sentinel an effective solution for students, developers, and small teams who need a lightweight monitoring tool that is easy to adapt and deploy.
The project validates that modern web tools can deliver powerful dashboards without unnecessary overhead.

Future Scope

The current version of Sentinel is solid, but several advanced improvements can be added:

- **Real-time data updates** using WebSockets or push-based cloud services.
- **Interactive charts and analytics** using libraries like Chart.js, ECharts, or D3.js.
- **Multi-user authentication system** with role-based access control through Google Cloud IAM or Firebase Auth.
- **Cloud-hosted deployment** using Google Cloud Run, Netlify, or Vercel.
- **Custom widgets and plugins** to extend dashboard functionality for different use cases.
- **Advanced logging and alerts**, enabling users to receive real-time notifications.
- **Dark/light theme switching** and more UI customization options.

This roadmap ensures that Sentinel can evolve into a full-featured cloud monitoring platform.